ServiceDesk Andina

Agente funcional para la recepción y asignación de incidentes de soporte empresarial

Integrantes Daniela Peña y Mirko Flores
Asignatura Ingeniería de Soluciones con IA · ISY0101
Entrega 6 de noviembre de 2026

Resumen ejecutivo

El proyecto implementa un agente de mesa de ayuda para ServiceDesk Andina, una empresa ficticia de servicios administrativos. Automatiza el triage de incidentes: consulta el caso, valida cobertura y SLA, calcula prioridad y genera una orden de atención simulada o deriva a revisión humana. Las decisiones quedan registradas y pueden recuperarse en sesiones posteriores.

Contexto y requerimiento organizacional

La recepción manual de fallas de equipos y aplicaciones obliga al personal a consultar información repetida, revisar cobertura y decidir quién atiende cada caso. Una asignación sin estas verificaciones puede comprometer recursos o ignorar una falla recurrente. El requerimiento es ejecutar ese flujo con reglas verificables y una explicación vinculada a políticas.

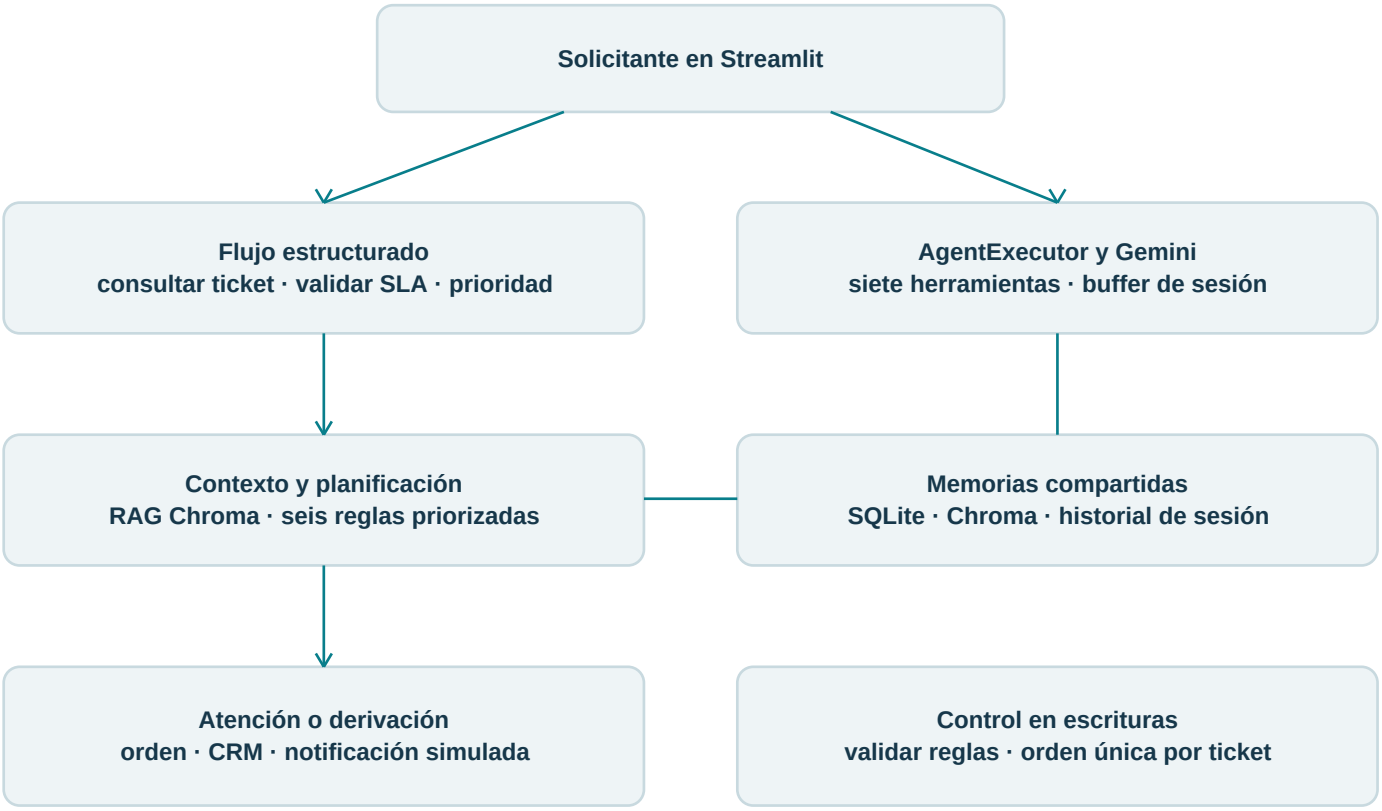
Se utilizan 40 incidentes y 15 solicitantes ficticios. El sistema registra órdenes y notificaciones locales; no compra repuestos, envía correos ni repara dispositivos. La solución demuestra automatización de atención y asignación, no una resolución técnica física ni una mejora empresarial medida.

Objetivos verificables

1. Integrar siete herramientas de consulta, cálculo, escritura y recuperación.
2. Mantener memoria de conversación, historial persistente y contexto semántico.
3. Cambiar el plan según cobertura, costo, recurrencia e información disponible.
4. Demostrar decisiones y acciones con pruebas y trazas reproducibles.

Arquitectura y orquestación

La implementación conserva la estructura técnica del tutorial y desarrolla un dominio distinto. Streamlit expone una ruta de atención estructurada y un asistente conversacional. Ambas utilizan herramientas LangChain; la ruta conversacional integra Gemini mediante AgentExecutor.



Contratos de las herramientas

consultar_ticket: recibe ID y devuelve incidente e historial.
validar_sla: devuelve elegibilidad, motivo y horas del acuerdo.
calcular_prioridad: devuelve P1, P2 o P3 y tiempo objetivo.
generar_orden: crea o recupera una orden mock, si las reglas lo permiten.
actualizar_crm: persiste un evento y rechaza estados incompatibles.
notificar_solicitante: guarda un mensaje local en canal mock.
buscar_casos_similares: recupera dos políticas y dos casos con fuente.

Los módulos se distribuyen en agent/tools, agent/memory, planner.py y core.py. Los scripts preparan CSV, SQLite y Chroma. Las consultas usan parámetros SQL y la clave se carga desde .env, excluido de Git.

Memoria y toma de decisiones

Tres mecanismos de memoria

Corto plazo. ConversationBufferMemory conserva mensajes en st.session_state. El prompt del agente recibe chat_history; la ruta estructurada guarda cada solicitud y resultado. Nueva conversación limpia ese buffer.
Largo plazo. SQLite conserva eventos, órdenes y notificaciones. consultar_ticket recupera decisiones anteriores y cuenta incidentes distintos atendidos en 90 días, sumados a antecedentes ficticios.
Semántica. MiniLM genera embeddings locales. Chroma contiene seis secciones de políticas y cuatro casos previos. La recuperación agrega texto y metadatos; las reglas estructuradas siguen siendo la autoridad de decisión.

Planificación priorizada

1. Cobertura vencida o SLA excedido → **escalar**.
2. Costo mayor a $100.000 → **solicitar_aprobacion**.
3. Tres o más incidentes en 90 días → **escalar_recurrencia**.
4. Historial, costo menor a $50.000 y descripción → **resolver_preferente**.
5. Descripción ausente → **solicitar_info**.
6. Resto de casos elegibles → **resolver_estandar**.

La primera regla satisfecha determina el plan. Las acciones resolver generan una orden de atención, no certifican una reparación. El SLA simulado es de 24 horas para impacto individual y 2 para impacto empresarial. La prioridad depende del impacto y la urgencia registrados.

Continuidad y adaptación

INC-001 pasa de atención estándar a preferente al recuperar el historial de la primera ejecución. La orden no se duplica. INC-035 solicita descripción y, al recibirla en una nueva solicitud, continúa hacia una orden. Los casos sin cobertura, recurrentes y de alto costo no generan órdenes automáticas.

Evaluación técnica del equipo

Completar con la justificación propia de los componentes y el criterio de prioridad utilizado.

Resultados y evidencias

Verificación del flujo

La suite automatizada registra 38 pruebas aprobadas. Verifica reglas y fronteras de montos, consulta SQL parametrizada, órdenes bloqueadas e idempotentes, persistencia, buffer, cambios de plan y conservación de firmas Gemini. Cada prueba utiliza SQLite temporal para evitar modificar la demostración. El archivo evidence/pytest_output.txt conserva la ejecución.

INC-001 · Atención estándar
Orden generada y notificación persistida. Al repetir, cambia a resolver_preferente.

INC-015 · Sin cobertura
La decisión es escalar; no se genera orden.

INC-022 · Alto costo
Costo de $250.000: solicitar_aprobacion, sin orden automática.

INC-030 · Recurrencia
Tres antecedentes activan revisión de causa raíz.

INC-035 · Información incompleta
Solicita descripción; al completarla permite la orden estándar.

INC-040 · SLA excedido
Impacto empresarial, prioridad P1 y escalamiento por tiempo excedido.

Recuperación y asistente conversacional

evidence/demo_resultados.json registra escenarios completos, fuentes recuperadas y tiempos por ejecución. La verificación real con Gemini 3.5 Flash completó dos turnos, utilizó consultar_ticket, validar_sla y buscar_casos_similares, y conservó cuatro mensajes en memoria. El resultado se registra en evidence/gemini_agent.json.

Reproducibilidad y límites

El README detalla entorno virtual, dependencias fijadas, CSV, carga SQLite, indexación y arranque de Streamlit. La recuperación requiere descargar MiniLM una vez; Gemini requiere conexión y cuota. Las órdenes y mensajes son locales. Autenticación, roles, autorizaciones posteriores y transacciones con outbox corresponden a una futura versión productiva.

Conclusiones y referencias

Espacios para la evaluación personal de los integrantes y sus aportes al proyecto. Completar antes de presentar y entregar.

Reflexión individual de Daniela Peña

Reflexión individual de Mirko Flores

Referencias

LangChain. (s. f.). *ChatGoogleGenerativeAI integration*. https://docs.langchain.com/oss/python/integrations/chat/google_generative_ai

LangChain. (s. f.). *Memory overview*. https://docs.langchain.com/oss/python/concepts/memory

deepset. (s. f.). *Agents*. https://docs.haystack.deepset.ai/docs/agents

Google. (s. f.). *Gemini models*. https://ai.google.dev/gemini-api/docs/models

Chroma. (s. f.). *Chroma documentation*. https://docs.trychroma.com/

Sentence Transformers. (s. f.). *all-MiniLM-L6-v2*. Hugging Face. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

Documentación consultada el 6 de noviembre de 2026. Los contratos, diagramas y detalles de correspondencia con la guía están disponibles en docs/.